

Listing 12.2 Making Monad a subtype of Applicative

```

trait Monad[F[_]] extends Applicative[F] {
  def flatMap[A,B](fa: F[A])(f: A => F[B]): F[B] = join(map(fa)(f))

  def join[A](ffa: F[F[A]]): F[A] = flatMap(ffa)(fa => fa)

  def compose[A,B,C](f: A => F[B], g: B => F[C]): A => F[C] =
    a => flatMap(f(a))(g)

  def map[B](fa: F[A])(f: A => B): F[B] =
    flatMap(fa)((a: A) => unit(f(a)))

  def map2[A,B,C](fa: F[A], fb: F[B])(f: (A, B) => C): F[C] =
    flatMap(fa)(a => map(fb)(b => f(a,b)))
}

```

A minimal implementation of Monad must implement unit and override either flatMap or join and map.



So far, we’ve been just rearranging the functions of our API and following the type signatures. Let’s take a step back to understand the difference in expressiveness between Monad and Applicative and what it all means.

12.3 The difference between monads and applicative functors

In the last chapter, we noted there were several minimal sets of operations that defined a Monad:

- unit and flatMap
- unit and compose
- unit, map, and join

Are the Applicative operations unit and map2 yet another minimal set of operations for monads? No. There are monadic combinators such as join and flatMap that can’t be implemented with just map2 and unit. To see convincing proof of this, take a look at join:

```

def join[A](f: F[F[A]]): F[A]

```

Just reasoning algebraically, we can see that unit and map2 have no hope of implementing this function. The join function “removes a layer” of *F*. But the unit function only lets us *add* an *F* layer, and map2 lets us apply a function *within F* but does no flattening of layers. By the same argument, we can see that Applicative has no means of implementing flatMap either.

So Monad is clearly adding some extra capabilities beyond Applicative. But what exactly? Let’s look at some concrete examples.